

# Configuration de l'ESP — Sentinel-X

Oct 8, 2026 · @Ma

Le boîtier Sentinel-X est un **NodeMCU ESP8266** (AZ-Delivery, puce USB CH340). Toute sa configuration tient dans deux fichiers placés à côté du sketch : `config.h` (identité, Wi-Fi, broker, seuils — secrets inclus) et `ca_cert.h` (certificat de la CA du groupe). Ce document regroupe ces fichiers, le câblage, les réglages de compilation et la procédure de flash. Les secrets y sont **masqués**.

## config.h

Fichier placé dans le dossier du sketch ( `Arduino/SentinelX/` ), **hors Git**. Il porte l'identité du boîtier, les accès Wi-Fi et broker, et les seuils. Les secrets ( `WIFI_PASS` , `MQTT_PASS` ) sont masqués ci-dessous.

```
#pragma once

// ----- Identité -----
#define GROUP_ID          "groupe1"           //
sentinel/<GROUP_ID>/<DEVICE_ID>/...
#define DEVICE_ID        "edge01"
#define DEVICE_HOSTNAME  "sentinel-edge01"    // nom réseau + client id MQTT

// ----- Wi-Fi (hotspot Windows « MATHIS ») -----
#define WIFI_SSID         "MATHIS"
#define WIFI_PASS         "●●●●●●●●"        // masqué

// ----- Broker MQTTS -----
#define MQTT_USER         "esp"
#define MQTT_PASS         "●●●●●●●●"        // masqué
#define MQTT_HOST         "192.168.137.1"    // IP du PC sur le hotspot
#define MQTT_USE_TLS      1                  // MQTTS, port 8883
#include "ca_cert.h"          // définit CA_CERT (voir section
suivante)

// ----- Heure -----
#define NTP_SERVER        "pool.ntp.org"
#define TZ_INFO           "CET-1CEST,M3.5.0,M10.5.0/3" // Europe/Paris

// ----- Capteurs / seuils -----
#define DHT_FAULT_COUNT   3                  // lectures DHT22 ratées avant alerte
capteur
#define TEMP_ALERT_C      50.0               // alerte température (°C)
```

```
#define TEMP_HYST_C      3.0      // hystérésis de fin d'alerte
#define PIR_WARMUP_S     30       // stabilisation du PIR après mise sous
tension
#define TELEMETRY_PERIOD_MS 2000  // période d'envoi (ms)

// ----- Fonctionnement -----
#define DEFAULT_ARMED    true     // détection d'intrusion active au
démarrage
#define BUZZER_ACTIVE_LOW false
```

À nettoyer : le `config.h` réel contient encore, en haut, un ancien bloc `static const char CA_CERT[] PROGMEM = ...` avant le `#include "ca_cert.h"`. C'est du code mort (double définition de `CA_CERT`) ; la CA réellement utilisée est celle de `ca_cert.h`. Le supprimer évite toute confusion pour un correcteur.

## ca\_cert.h

Certificat de la CA du groupe (« Sentinel-X CA »), celle qui a signé le certificat du broker. Le firmware l'épingle via `X509List(CA_CERT)` : seul un serveur présentant un certificat signé par cette CA est accepté.

Le format est un `#define` **multi-lignes**, une ligne PEM par chaîne `"...\n"` avec continuation `\`. Ce format est imposé par le préprocesseur : une *raw string* `R"PEM(...)"` dans un `#define` ne compile pas (le préprocesseur termine la macro au premier retour à la ligne).

```
// CA Sentinel-X - généré automatiquement, ne pas éditer
#define CA_CERT \
"-----BEGIN CERTIFICATE-----\n" \
"MIIBYTCCA+WgAwIBAgIUUTIP/+XdUttVpR1DfK3W+Xmrs24UwCgYIKoZIzj0EAwIw\n" \
"      ... (corps du certificat tronqué) ...      \n" \
"-----END CERTIFICATE-----\n"
```

Caractéristiques de la CA :

- Sujet : `C=FR, O=Sentinel-X, CN=Sentinel-X CA` (auto-signée, usage interne au groupe).
- Clé EC P-256, signature ECDSA-SHA256.
- Générée automatiquement à partir de `ca.crt` (openssl) ; **hors Git**.

## Sketch SentinelX.ino — structure et bibliothèques

Le sketch (~750 lignes, firmware v2.8.2) s'organise en deux temps :

- **setup()** : port série, bus I2C + écran LCD, capteur DHT22, puis TLS ( `X509List(CA_CERT)` + `setTrustAnchors` ), connexion Wi-Fi, synchronisation de l'heure (NTP), connexion MQTTS et abonnement aux topics `.../cmd` et `.../config`.
- **loop()** : lecture des capteurs (DHT22, PIR, MQ-2), rafraîchissement de l'écran, publication de la télémétrie toutes les 2 s, traitement des commandes/réglages reçus, gestion du buzzer.

Topics utilisés : `sentinel/groupe1/edge01/` + `telemetry` | `alert` | `status` | `cmd` | `config`.

| Bibliothèque                  | Rôle                                           | Version                    |
|-------------------------------|------------------------------------------------|----------------------------|
| ESP8266 core (Arduino)        | carte NodeMCU, Wi-Fi, TLS BearSSL              | 3.1.2                      |
| PubSubClient (knolleary)      | client MQTT                                    | dernière (Library Manager) |
| ArduinoJson                   | (dé)sérialisation JSON<br>télémétrie/commandes | v7                         |
| LiquidCrystal_I2C             | écran LCD 16x2 via module I2C                  | dernière (Library Manager) |
| DHT sensor library (Adafruit) | capteur DHT22<br>(température/humidité)        | dernière (Library Manager) |

## Carte et réglages de compilation

- **Carte** : NodeMCU 1.0 (module ESP-12E), puce USB **CH340** (pilote CH340 requis sous Windows).
- **Core** : esp8266 by ESP8266 Community, **v3.1.2**.
- **FQBN** : `esp8266:esp8266:nodemcu2`.
- **Vitesse de téléversement** : 115200 bauds ; taille de flash par défaut (4 Mo).
- **Port** : `COMx` sous Windows — apparaît quand l'ESP est branché en USB.

Compilation en ligne de commande (arduino-cli), depuis le dossier du sketch :

```
arduino-cli compile --fqbn esp8266:esp8266:nodemcu2 .
```

## Câblage

| Composant               | Signal → broche              | Alimentation     | Remarque                                      |
|-------------------------|------------------------------|------------------|-----------------------------------------------|
| Écran LCD 16x2 (I2C)    | SDA → D2, SCL → D1           | 5V               | adresse I2C détectée auto (0x27 / 0x3F)       |
| DHT22 (temp./hum.)      | OUT → D5                     | 3,3V             | —                                             |
| PIR HC-SR501 (présence) | OUT → D6                     | 5V               | stabilisation 30 s au démarrage               |
| Buzzer                  | S → D8                       | 3,3V             | buzzer passif, piloté par <code>tone()</code> |
| MQ-2 (fumée/gaz)        | A0 → A0                      | 5V               | via pont diviseur 10k/20k                     |
| NodeMCU                 | GND → rail – (masse commune) | USB / power bank | ne jamais relier 3V ou VIN à une alim externe |

**Note** : le buzzer est sur **D8** dans le firmware ( `PIN_BUZZER = D8` ). Le libellé « D7 » affiché dans le dashboard est purement cosmétique (reste d'un ancien patch) et n'a aucun effet sur le câblage réel.

## Procédure de flash

1. Brancher le NodeMCU en USB : un port `COM` apparaît (ex. `COM5` ). Si rien n'apparaît, c'est le pilote CH340 qui manque, ou un câble USB « charge seule » (prendre un câble data).
2. Placer `config.h` et `ca_cert.h` dans le dossier du sketch.
3. Compiler puis téléverser avec `arduino-cli` :

```
arduino-cli compile --fqbn esp8266:esp8266:nodemcu2 .
arduino-cli upload --fqbn esp8266:esp8266:nodemcu2 -p COM5 .
```

4. Vérifier au moniteur série (115200 bauds) : connexion Wi-Fi, heure NTP, puis `[mqtt]` connecté .

**Alimentation pour la démo** : alimenter le boîtier par **USB / power bank** (micro-USB). Une alimentation de breadboard (MB102) est trop faible pour les pics de courant du Wi-Fi et provoque un *brownout* (écran blanc, pas de connexion).

## Code source complet (SentinelX.ino)

Firmware v2.8.2, intégral. Partie 1/2 — en-tête, includes, configuration par défaut, broches, état, outils, MQTT, réseau (Wi-Fi → heure → MQTTS).

```
// =====
//  Sentinel-X · firmware du boîtier (Edge Node) · v2.7.0
//
//  Matériel : NodeMCU ESP8266 (AZ-Delivery, CH340), écran LCD 16x2 I2C,
//              DHT22, détecteur de mouvement HC-SR501, capteur de fumée MQ-2,
//  buzzer
//
//  Câblage :
//  Écran LCD    GND -> rail -    VCC -> rail 5V    SDA -> D2    SCL -> D1
//  DHT22        + -> rail 3,3V    out -> D5    - -> rail -
//  Détecteur    VCC -> rail 5V (côté diode)    OUT -> D6    GND -> rail -
//  Buzzer       S -> D8    + -> rail 3,3V    - -> rail -
//  MQ-2         VCC -> rail 5V    GND -> rail -    A0 -> pont 10k/20k -> A0
//  D0 -> rien
//  NodeMCU      G -> rail -    (ne jamais relier 3V ou VIN au module
//  d'alimentation)
//
//  Ce que fait le boîtier (consignes Sentinel-X) :
//  - lit température, humidité, présence et fumée/gaz ;
//  - affiche tout sur l'écran LCD ;
//  - envoie les mesures au serveur en MQTT toutes les 2 s :
//      MQTTS chiffré (port 8883, MQTT_USE_TLS 1) ou MQTT simple (port 1883,
//  MQTT_USE_TLS 0) ;
//  - publie chaque alerte (fumée/gaz, température, intrusion, capteur en
//  panne) ;
//  - exécute les commandes du dashboard (armer, acquitter, buzzer,
//  message) ;
//  - publie "online" à la connexion, le broker publie "offline" s'il
//  disparaît.
//
//  Topics : TOPIC_BASE/telemetry | alert | status | cmd (par défaut
//  sentinel/<groupe>/<boîtier>)
//  Réglages et secrets : dans config.h (à ne jamais mettre dans Git).
//  =====

#include <Arduino.h>
#include <ESP8266WiFi.h>
#include <WiFiClientSecure.h>
#include <PubSubClient.h>
#include <ArduinoJson.h>
#include <Wire.h>
```

```

#include <LiquidCrystal_I2C.h>
#include <DHT.h>
#include <time.h>
#include <math.h>

#if __has_include("config.h")
#include "config.h"
#else
#error "Il manque config.h dans le dossier du sketch."
#endif

// ----- Valeurs par défaut si config.h ne les définit pas -----
#if !defined(WIFI_PASS) && defined(WIFI_PASSWORD)
#define WIFI_PASS WIFI_PASSWORD // accepte les deux noms
#endif
#if !defined(WIFI_SSID) || !defined(WIFI_PASS) || !defined(MQTT_HOST) ||
!defined(MQTT_PASS)
#error "config.h doit définir WIFI_SSID, WIFI_PASS (ou WIFI_PASSWORD),
MQTT_HOST et MQTT_PASS."
#endif
#ifndef GROUP_ID
#define GROUP_ID "g1"
#endif
#ifndef DEVICE_ID
#define DEVICE_ID "edge01"
#endif
#ifndef DEVICE_HOSTNAME
#define DEVICE_HOSTNAME "sentinel-edge-01"
#endif
#ifndef MQTT_USE_TLS
#define MQTT_USE_TLS 1 // 1 = MQTTS chiffré (8883), 0 = MQTT
simple (1883)
#endif
#ifndef MQTT_PORT
#define MQTT_PORT (MQTT_USE_TLS ? 8883 : 1883)
#endif
#ifndef MQTT_USER
#define MQTT_USER "esp"
#endif
#ifndef NTP_SERVER
#define NTP_SERVER MQTT_HOST // le serveur donne l'heure
#endif
#ifndef TZ_INFO
#define TZ_INFO "CET-1CEST,M3.5.0,M10.5.0/3"
#endif
#ifndef BUZZER_ACTIVE_LOW

```

```

#define BUZZER_ACTIVE_LOW false
#endif
#ifndef DEFAULT_MOTION_BUZZER
#define DEFAULT_MOTION_BUZZER false // buzzer sur détection de mouvement
(réglable depuis le dashboard)
#endif
#ifndef DEFAULT_ARMED
#define DEFAULT_ARMED true
#endif
#ifndef TELEMETRY_PERIOD_MS
#define TELEMETRY_PERIOD_MS 2000
#endif
#ifndef PIR_WARMUP_S
#define PIR_WARMUP_S 60
#endif
#ifndef TEMP_ALERT_C
#define TEMP_ALERT_C 45.0
#endif
#ifndef TEMP_HYST_C
#define TEMP_HYST_C 2.0
#endif
#ifndef DHT_FAULT_COUNT
#define DHT_FAULT_COUNT 3
#endif
#ifndef LCD_ADDR
#define LCD_ADDR 0 // 0 = détection automatique de l'adresse
de l'écran
#endif
#ifndef TOPIC_BASE
#define TOPIC_BASE "sentinel/" GROUP_ID "/" DEVICE_ID // préfixe des topics
MQTT
#endif
#ifndef USE_MQ2
#define USE_MQ2 1 // capteur de fumée MQ-2 branché sur A0
#endif
#ifndef MQ2_WARMUP_S
#define MQ2_WARMUP_S 60 // chauffe du MQ-2 avant d'exploiter la
mesure
#endif
#ifndef GAS_ALERT_RAW
#define GAS_ALERT_RAW 600 // valeur A0 (0-1023) qui déclenche
l'alerte
#endif
#ifndef GAS_CLEAR_RAW
#define GAS_CLEAR_RAW 500 // retour à la normale en dessous
(hystérésis)

```

```

#endif

#define FW_VERSION "2.8.2"

// ----- Broches -----
static const uint8_t PIN_SCL      = D1;    // écran LCD
static const uint8_t PIN_SDA      = D2;    // écran LCD
static const uint8_t PIN_DHT      = D5;    // DHT22
static const uint8_t PIN_PIR      = D6;    // détecteur de mouvement
static const uint8_t PIN_BUZZER   = D8;    // buzzer
static const uint8_t PIN_GAS      = A0;    // MQ-2 (sortie A0 du module, via le
pont diviseur)

// ----- Réglages internes -----
static const uint32_t LCD_PERIOD_MS    = 500;
static const uint32_t LCD_PAGE_MS      = 3000;
static const uint32_t PIR_POLL_MS      = 50;
static const uint32_t MQTT_BACKOFF_MIN = 2000;
static const uint32_t MQTT_BACKOFF_MAX = 30000;
static const unsigned CMD_MAX_LEN      = 256;
static const time_t   VALID_TIME       = 17000000000;    // en dessous, l'heure
n'est pas encore réglée
static const uint32_t GAS_PERIOD_MS    = 1000;
static const uint32_t DHT_PERIOD_MS    = 2500;    // le DHT22 a besoin d'au
moins 2 s entre deux lectures
static const uint8_t  DHT_FAULT_LIMIT  = (DHT_FAULT_COUNT > 6) ?
DHT_FAULT_COUNT : 6;    // ~15 s de lectures ratées

// ----- Objets -----
#if MQTT_USE_TLS
BearSSL::WiFiClientSecure net;    // connexion chiffrée, certificat vérifié
BearSSL::X509List *caCert = nullptr;
#else
WiFiClient net;    // connexion MQTT simple, non chiffrée
#endif
PubSubClient mqtt(net);
DHT dht(PIN_DHT, DHT22);
LiquidCrystal_I2C *lcd = nullptr;

// ----- Topics -----
char topicTelemetry[64], topicAlert[64], topicStatus[64], topicCmd[64],
topicConfig[64];

// ----- État -----
float temperature = NAN;
float humidity = NAN;

```

```

uint8_t dhtFailures = 0;
bool motion = false;
int gasRaw = -1;                                // -1 = pas encore de mesure

bool alertGas = false;
bool alertTemp = false;
bool alertMotion = false;
bool alertSensor = false;
bool remoteAlarm = false;                      // alarme déclenchée par le serveur (ex.
caméra IA)
uint32_t remoteAlarmUntil = 0;
char remoteReason[13] = "CAMERA";
bool armed = DEFAULT_ARMED;
bool silenced = false;                        // alarme acquittée : le buzzer se tait
bool buzzOnMotion = DEFAULT_MOTION_BUZZER;    // le buzzer sonne-t-il quand le
PIR détecte ? (dashboard)

bool buzzerManual = false;                    // commande "buzzer" reçue du dashboard
bool buzzerManualFast = false;
uint32_t buzzerManualUntil = 0;

char lcdMessage[17] = "";                    // message envoyé depuis le dashboard
uint32_t lcdMessageUntil = 0;

uint32_t lastSensorRead = 0, lastDhtRead = 0, lastGasRead = 0, lastLcd = 0,
lastPirPoll = 0;
uint32_t dhtOk = 0, dhtBad = 0;              // compteurs pour le diagnostic
uint32_t lastMqttAttempt = 0, lastNtpLog = 0;
uint32_t wifiUpSince = 0;
bool timeFallbackUsed = false;
static const uint32_t NTP_WAIT_MS = 20000;    // au-delà, on prend l'heure de
compilation
uint32_t mqttBackoff = MQTT_BACKOFF_MIN;
uint32_t telemetrySeq = 0;
bool wifiWasUp = false, mqttWasUp = false;

// =====
// Outils
// =====
static bool notExpired(uint32_t until) { return (int32_t)(until - millis()) >
0; }
static double round1(float v) { return round(v * 10.0) / 10.0; }
static bool timeIsValid() { return time(nullptr) > VALID_TIME; }
static bool pirReady() { return millis() >= PIR_WARMUP_S * 1000UL; }
static bool gasReady() { return USE_MQ2 && millis() >= MQ2_WARMUP_S * 1000UL;
}

```

```
// Heure de compilation du firmware (ex. "Oct 6 2026" "08:39:12"), en heure
locale
static time_t buildTime() {
    static const char months[] = "JanFebMarAprMayJunJulAugSepOctNovDec";
    char m[4] = "";
    int d = 1, y = 2026, hh = 0, mm = 0, ss = 0;
    sscanf(__DATE__, "%3s %d %d", m, &d, &y);
    sscanf(__TIME__, "%d:%d:%d", &hh, &mm, &ss);
    const char *pos = strstr(months, m);
    struct tm t = {};
    t.tm_year = y - 1900;
    t.tm_mon = pos ? (pos - months) / 3 : 0;
    t.tm_mday = d;
    t.tm_hour = hh;
    t.tm_min = mm;
    t.tm_sec = ss;
    t.tm_isdst = -1;
    return mktime(&t);
}

// Garde uniquement l'ASCII imprimable (l'écran n'affiche pas les accents)
static void sanitize(const char *src, char *dst, size_t size, bool upper) {
    size_t j = 0;
    for (size_t i = 0; src[i] && j + 1 < size; i++) {
        char c = src[i];
        if (c < 0x20 || c > 0x7E) continue;
        if (upper && c >= 'a' && c <= 'z') c -= 32;
        dst[j++] = c;
    }
    dst[j] = '\0';
}

static bool i2cPresent(uint8_t addr) {
    Wire.beginTransmission(addr);
    return Wire.endTransmission() == 0;
}

#ifdef BUZZER_PASSIVE
#define BUZZER_PASSIVE 1 // 1 = buzzer passif : on lui envoie une
fréquence (beaucoup plus fort)
#endif
#ifdef BUZZER_FREQ
#define BUZZER_FREQ 2700 // Hz : fréquence de résonance typique, là où
il sonne le plus fort
#endif
```

```

static void buzzerWrite(bool on) {
#if BUZZER_PASSIVE
    static bool playing = false;
    if (on == playing) return;    // appelée à chaque tour de boucle : on ne
    relance pas la note
    playing = on;
    if (on) tone(PIN_BUZZER, BUZZER_FREQ);
    else { noTone(PIN_BUZZER); digitalWrite(PIN_BUZZER, LOW); }
#else
    digitalWrite(PIN_BUZZER, (on != BUZZER_ACTIVE_LOW) ? HIGH : LOW);
#endif
}

// =====
// MQTT : envoi des données
// =====
void publishTelemetry() {
    if (!mqtt.connected()) return;

    JsonDocument doc;
    doc["device"] = DEVICE_ID;
    doc["fw"] = FW_VERSION;
    doc["ts"] = (uint32_t)time(nullptr);
    doc["seq"] = ++telemetrySeq;
    if (isnan(temperature)) doc["temperature"] = nullptr; else
doc["temperature"] = round1(temperature);
    if (isnan(humidity)) doc["humidity"] = nullptr; else doc["humidity"] =
round1(humidity);
    if (!USE_MQ2 || gasRaw < 0) doc["gas_raw"] = nullptr; else doc["gas_raw"] =
gasRaw;
    if (USE_MQ2) doc["gas_ready"] = gasReady(); else doc["gas_ready"] =
nullptr;
    doc["motion"] = motion;
    doc["armed"] = armed;
    doc["silenced"] = silenced;
    doc["motion_buzzer"] = buzzOnMotion;

    JsonObject a = doc["alerts"].to<JsonObject>();
    a["gas"] = alertGas;
    a["temperature"] = alertTemp;
    a["motion"] = alertMotion;
    a["sensor_fault"] = alertSensor;
    a["remote"] = remoteAlarm;

    doc["rssi"] = WiFi.RSSI();

```

```

    doc["uptime"] = millis() / 1000;
    doc["heap"] = ESP.getFreeHeap();

    char buf[480];
    size_t n = serializeJson(doc, buf, sizeof buf);
    mqtt.publish(topicTelemetry, (const uint8_t *)buf, n, false);
}

void publishAlert(const char *type, bool raised, const char *severity, float
value, float threshold) {
    Serial.printf("[alerte] %s %s\n", type, raised ? "levée" : "terminée");
    if (!mqtt.connected()) return;    // l'état reste visible dans la
télémetrie suivante

    JsonDocument doc;
    doc["device"] = DEVICE_ID;
    doc["ts"] = (uint32_t)time(nullptr);
    doc["type"] = type;
    doc["state"] = raised ? "raised" : "cleared";
    doc["severity"] = severity;
    if (!isnan(value)) doc["value"] = round1(value);
    if (!isnan(threshold)) doc["threshold"] = round1(threshold);
    doc["armed"] = armed;

    char buf[256];
    size_t n = serializeJson(doc, buf, sizeof buf);
    mqtt.publish(topicAlert, (const uint8_t *)buf, n, false);
}

// =====
// Réseau : Wi-Fi -> heure -> MQTTS
// =====
bool mqttConnect() {
    Serial.printf("[mqtt] connexion à %s:%d (%s)...\n", MQTT_HOST,
(int)MQTT_PORT, MQTT_USE_TLS ? "TLS" : "sans TLS");
    #if MQTT_USE_TLS
        net.setX509Time(time(nullptr));    // l'heure sert à vérifier le certificat
du serveur
    #endif

    bool ok = mqtt.connect(DEVICE_HOSTNAME, MQTT_USER, MQTT_PASS,
                            topicStatus, 1, true, "offline", true);    //
"offline" publié par le broker si on disparaît
    if (!ok) {
        Serial.printf("[mqtt] échec, état %d", mqtt.state());
    }
    #if MQTT_USE_TLS

```

```

    char err[96] = "";
    int code = net.getLastSSLError(err, sizeof err);
    if (code) Serial.printf(", TLS %d : %s", code, err);
#endif
    Serial.println(mqtt.state() == 5 ? " (identifiants refusés par le
broker)" :
                    mqtt.state() == -2 ? " (broker injoignable : IP, port ou
pare-feu)" : "");
    return false;
}
mqtt.publish(topicStatus, "online", true);
mqtt.subscribe(topicCmd, 1);
mqtt.subscribe(topicConfig, 1);    // réglages conservés par le broker
(retained)
Serial.printf("[mqtt] connecté, envoi sur %s\n", topicTelemetry);
publishTelemetry();
return true;
}

void netTask(uint32_t now) {
    bool wifiUp = WiFi.status() == WL_CONNECTED;
    if (wifiUp != wifiWasUp) {
        wifiWasUp = wifiUp;
        if (wifiUp) {
            Serial.printf("[wifi] connecté, IP %s\n",
WiFi.localIP().toString().c_str());
            // Heure : d'abord le serveur, puis Internet si le point d'accès le
partage
            configTime(TZ_INFO, NTP_SERVER, "pool.ntp.org", "time.windows.com");
            wifiUpSince = now;
            mqttBackoff = MQTT_BACKOFF_MIN;
        } else {
            Serial.println(F("[wifi] déconnecté, reconnexion automatique"));
        }
    }
    static uint32_t lastWifiLog = 0;
    if (!wifiUp && now - lastWifiLog > 10000) {
        lastWifiLog = now;
        int st = WiFi.status();
        Serial.printf("[wifi] toujours pas connecté à %s (%s)\n", WIFI_SSID,
                        st == WL_NO_SSID_AVAIL ? "réseau introuvable : il doit être
en 2,4 GHz" :
                        st == WL_CONNECT_FAILED ? "mot de passe refusé" :
"connexion en cours");
    }
}

```

```

bool mqttUp = mqtt.connected();
if (mqttWasUp && !mqttUp) {
    Serial.printf("[mqtt] connexion perdue (état %d, Wi-Fi %d dBm, mémoire
libre %u)\n",
                    mqtt.state(), WiFi.RSSI(), ESP.getFreeHeap());
    mqttBackoff = MQTT_BACKOFF_MIN;    // on retente vite
}
mqttWasUp = mqttUp;
if (!wifiUp || mqttUp) return;

if (MQTT_USE_TLS && !timeIsValid()) {    // sans TLS, pas besoin d'attendre
l'heure
    if (now - wifiUpSince > NTP_WAIT_MS && !timeFallbackUsed) {
        // Aucun serveur d'heure joignable (réseau d'école filtré) : sans
heure, le certificat
        // serait jugé "pas encore valide". On part de l'heure de compilation,
la synchro NTP
        // continue en arrière-plan et corrigera l'heure dès qu'un serveur
répond.
        timeFallbackUsed = true;
        struct timeval tv = { buildTime(), 0 };
        settimeofday(&tv, nullptr);
        Serial.println(F("[ntp] aucun serveur d'heure : heure de compilation
utilisée"));
    } else if (now - lastNtpLog > 5000) {
        lastNtpLog = now;
        Serial.printf("[ntp] attente de l'heure depuis %s\n", NTP_SERVER);
        return;
    }
    if (!timeIsValid()) return;
}

if (now - lastMqttAttempt < mqttBackoff) return;
lastMqttAttempt = now;
if (mqttConnect()) {
    mqttBackoff = MQTT_BACKOFF_MIN;
    mqttWasUp = true;
} else {
    mqttBackoff = min(mqttBackoff * 2, MQTT_BACKOFF_MAX);
}
}

```

Partie 2/2 — capteurs et alertes (DHT22, MQ-2, PIR), commandes du dashboard, écran LCD, buzzer, démarrage ( `setup` ) et boucle principale ( `loop` ).

```
// =====
//  Capteurs et alertes
// =====
void readDht() {
    float t = dht.readTemperature();
    float h = dht.readHumidity();

    // Une lecture n'est retenue que si elle est plausible : hors plage ou saut
    brutal = parasite
    const char *why = nullptr;
    if (isnan(t) || isnan(h)) why = "pas de réponse";
    else if (t < -40 || t > 80 || h < 0 || h > 100) why = "valeur hors plage";
    else if (!isnan(temperature) && (fabsf(t - temperature) > 8 || fabsf(h -
humidity) > 25)) why = "saut brutal";

    if (why) {
        dhtBad++;
        if (dhtFailures < 255) dhtFailures++;
        if (dhtFailures >= DHT_FAULT_LIMIT) {           // seulement après ~15 s
sans lecture valable
            temperature = NAN;
            humidity = NAN;
        }
        // Un "saut brutal" répété 3 fois de suite est une vraie variation : on
l'accepte
        if (!strcmp(why, "saut brutal") && dhtFailures >= 3) {
            temperature = t;
            humidity = h;
            dhtFailures = 0;
        }
    } else {
        dhtOk++;
        temperature = t;
        humidity = h;
        dhtFailures = 0;
    }

    static uint32_t lastDhtLog = 0;
    if (millis() - lastDhtLog > 10000) {
        lastDhtLog = millis();
        if (why) Serial.printf("[dht] lecture rejetée (%s), %u ok / %u rejetées
depuis le démarrage\n", why, dhtOk, dhtBad);
        else Serial.printf("[dht] %.1f °C  %.0f %%  (%u ok / %u rejetées)\n", t,
h, dhtOk, dhtBad);
    }
}
```

```

// Température trop haute (départ de feu), avec hystérésis
if (!isnan(temperature)) {
    if (!alertTemp && temperature >= TEMP_ALERT_C) {
        alertTemp = true;
        silenced = false;
        publishAlert("temperature", true, "critical", temperature,
TEMP_ALERT_C);
    } else if (alertTemp && temperature < TEMP_ALERT_C - TEMP_HYST_C) {
        alertTemp = false;
        publishAlert("temperature", false, "critical", temperature,
TEMP_ALERT_C);
    }
}

// DHT22 en panne ou débranché
bool fault = dhtFailures >= DHT_FAULT_LIMIT;
if (fault != alertSensor) {
    alertSensor = fault;
    publishAlert("sensor_fault", fault, "warning", NAN, NAN);
}

// Fin de l'alarme envoyée par le serveur
if (remoteAlarm && !notExpired(remoteAlarmUntil)) remoteAlarm = false;
}

// MQ-2 : moyenne de 8 mesures, puis alerte avec hystérésis une fois le
capteur chaud
void readGas() {
    if (!USE_MQ2) return;
    uint32_t sum = 0;
    for (uint8_t i = 0; i < 8; i++) {
        sum += analogRead(PIN_GAS);
        delay(2);
    }
    gasRaw = sum / 8;

    static uint32_t lastGasLog = 0;
    if (millis() - lastGasLog > 10000) {
        lastGasLog = millis();
        if (gasReady()) Serial.printf("[gaz] %d (alerte à %d)\n", gasRaw,
GAS_ALERT_RAW);
        else Serial.printf("[gaz] %d (chauffe en cours, encore %lu s)\n", gasRaw,
(unsigned long)((MQ2_WARMUP_S * 1000UL - millis()) /
1000 + 1));
    }
}

```

```

    if (!gasReady()) return;
    if (!alertGas && gasRaw >= GAS_ALERT_RAW) {
        alertGas = true;
        silenced = false;
        publishAlert("gas", true, "critical", gasRaw, GAS_ALERT_RAW);
    } else if (alertGas && gasRaw < GAS_CLEAR_RAW) {
        alertGas = false;
        publishAlert("gas", false, "critical", gasRaw, GAS_ALERT_RAW);
    }
}

void pirTask(uint32_t now) {
    if (now - lastPirPoll < PIR_POLL_MS) return;
    lastPirPoll = now;

    bool raw = digitalRead(PIN_PIR) == HIGH;
    static int8_t lastRaw = -1;
    if (raw != lastRaw) {
        // trace brute pour vérifier le
câblage
        lastRaw = raw;
        Serial.printf("[pir] signal %s%s\n", raw ? "HAUT (mouvement)" : "BAS",
            pirReady() ? "" : " (stabilisation en cours, ignoré)");
    }

    bool m = pirReady() && raw;
    if (m == motion) return;
    motion = m;
    alertMotion = m;
    if (m && armed) silenced = false;
    publishAlert("motion", m, armed ? "warning" : "info", NAN, NAN);
}

// =====
// Commandes reçues du dashboard (topic .../cmd)
// {"action":"arm"} {"action":"disarm"} {"action":"ack"}
{"action":"ping"}
// {"action":"buzzer","state":"on|off|beep","duration_ms":2000}
//
{"action":"alarm","state":"on|off","reason":"camera","duration_ms":30000}
// {"action":"display","text":"Bonjour jury","duration_ms":10000}
// {"action":"motion_buzzer","state":"on|off"}
// Réglages (topic .../config, retained) : {"motion_buzzer": true|false}
// =====
static void setMotionBuzzer(bool on) {
    if (on == buzzOnMotion) return;
    buzzOnMotion = on;
}

```

```

    Serial.printf("[config] buzzer sur détection : %s\n", on ? "activé" :
"désactivé");
}

void onMqttMessage(char *topic, byte *payload, unsigned int length) {
    if (length == 0 || length > CMD_MAX_LEN) return;
    if (strcmp(topic, topicConfig) == 0) {    // réglages envoyés par le
dashboard
        JsonDocument cfg;
        if (deserializeJson(cfg, payload, length)) return;
        if (cfg["motion_buzzer"].is<bool>())
setMotionBuzzer(cfg["motion_buzzer"].as<bool>());
        publishTelemetry();
        return;
    }
    if (strcmp(topic, topicCmd) != 0) return;

    JsonDocument doc;
    if (deserializeJson(doc, payload, length)) return;    // JSON invalide :
ignoré

    const char *action = doc["action"] | "";
    const char *state = doc["state"] | "";
    uint32_t now = millis();
    Serial.printf("[cmd] %s %s\n", action, state);

    if (!strcmp(action, "arm")) {
        armed = true;
    } else if (!strcmp(action, "disarm")) {
        armed = false;
    } else if (!strcmp(action, "ack")) {
        silenced = true;
    } else if (!strcmp(action, "ping")) {
        // rien : la télémétrie part juste en dessous
    } else if (!strcmp(action, "buzzer")) {
        long dur = constrain(doc["duration_ms"] | 1000L, 100L, 10000L);
        if (!strcmp(state, "on") || !strcmp(state, "beep")) {
            buzzerManual = true;
            buzzerManualFast = !strcmp(state, "beep");
            buzzerManualUntil = now + dur;
        } else if (!strcmp(state, "off")) {
            buzzerManual = false;
            silenced = true;
        } else return;
    } else if (!strcmp(action, "alarm")) {
        if (!strcmp(state, "on")) {

```

```

        long dur = constrain(doc["duration_ms"] | 30000L, 1000L, 600000L);
        sanitize(doc["reason"] | "camera", remoteReason, sizeof remoteReason,
true);
        remoteAlarm = true;
        remoteAlarmUntil = now + dur;
        silenced = false;
    } else if (!strcmp(state, "off")) {
        remoteAlarm = false;
    } else return;
} else if (!strcmp(action, "motion_buzzer")) {
    if (!strcmp(state, "on")) setMotionBuzzer(true);
    else if (!strcmp(state, "off")) setMotionBuzzer(false);
    else return;
} else if (!strcmp(action, "display")) {
    long dur = constrain(doc["duration_ms"] | 10000L, 1000L, 60000L);
    sanitize(doc["text"] | "", lcdMessage, sizeof lcdMessage, false);
    lcdMessageUntil = now + dur;
} else {
    return; // action inconnue ou non gérée par ce
boîtier
}
publishTelemetry(); // le dashboard voit le changement tout
de suite
}

// =====
// Écran LCD 16x2
// =====
static void lcdLine(uint8_t row, const char *text) {
    char buf[17];
    snprintf(buf, sizeof buf, "%-16.16s", text); // complète avec des espaces
: pas de scintillement
    lcd->setCursor(0, row);
    lcd->print(buf);
}

// Ligne d'alerte, renvoie false s'il n'y a rien à signaler
static bool alertText(char *out, size_t size) {
    if (alertGas) { snprintf(out, size, silenced ? "Gaz: acquitte"
: "!! ALERTE GAZ !!"); return true; }
    if (alertTemp) { snprintf(out, size, silenced ? "Temp: acquitte"
: "!! TEMP ELEVEE !"); return true; }
    if (remoteAlarm) { snprintf(out, size, "! ALERTE %.7s",
remoteReason); return true; }
    // Intrusion : plus affichée sur l'écran (toujours envoyée au serveur)
    if (alertSensor) { snprintf(out, size, "Capteur DHT22 HS"); return

```

```

true; }
    if (lcdMessage[0] && notExpired(lcdMessageUntil)) { snprintf(out, size,
"%s", lcdMessage); return true; }
    return false;
}

void lcdTask(uint32_t now) {
    if (!lcd || now - lastLcd < LCD_PERIOD_MS) return;
    lastLcd = now;

    // Ligne du haut, toujours affichée : valeurs du DHT22, ex. "T 26.3°C H
54%"
    char l0[32], l1[32], t[8], h[6], alert[24];
    if (isnan(temperature)) snprintf(t, sizeof t, "--.-"); else snprintf(t,
sizeof t, "%.1f", temperature);
    if (isnan(humidity)) snprintf(h, sizeof h, "--"); else snprintf(h, sizeof
h, "%d", (int)lroundf(humidity));
    snprintf(l0, sizeof l0, "T %s%cC H %s%%", t, (char)0xDF, h); // 0xDF =
symbole degré
    lcdLine(0, l0);

    // Ligne du bas : alerte s'il y en a une, sinon alternance présence /
connexion
    if (alertText(alert, sizeof alert)) {
        lcdLine(1, alert);
    } else if (USE_MQ2 && (now / LCD_PAGE_MS) % 3 == 2) { // page fumée/gaz
        if (!gasReady()) snprintf(l1, sizeof l1, "Gaz chauffe %lus", (unsigned
long)((MQ2_WARMUP_S * 1000UL - now) / 1000 + 1));
        else if (gasRaw < 0) snprintf(l1, sizeof l1, "Gaz --");
        else snprintf(l1, sizeof l1, "Gaz %d/%d", gasRaw, GAS_ALERT_RAW);
        lcdLine(1, l1);
    } else if ((now / LCD_PAGE_MS) % (USE_MQ2 ? 3 : 2) == 0) {
        if (!pirReady()) snprintf(l1, sizeof l1, "PIR pret ds %lus", (unsigned
long)((PIR_WARMUP_S * 1000UL - now) / 1000 + 1));
        else snprintf(l1, sizeof l1, "PIR %-4s %s", motion ? "oui" : "non", armed
? "Arme" : "Desarme");
        lcdLine(1, l1);
    } else {
        if (WiFi.status() != WL_CONNECTED) snprintf(l1, sizeof l1, "WiFi:
recherche");
        else snprintf(l1, sizeof l1, "Serveur: %s", mqtt.connected() ? "OK" : "--
");
        lcdLine(1, l1);
    }
}
}

```

```
// =====
// Buzzer
// =====
void buzzerTask(uint32_t now) {
    if (buzzerManual && !notExpired(buzzerManualUntil)) buzzerManual = false;

    bool on = false;
    if (buzzerManual) on = buzzerManualFast ? (now / 150) % 2 == 0 : true;
    else if (!silenced && (alertGas || alertTemp || remoteAlarm)) on = (now /
150) % 2 == 0;    // rapide : critique
    else if (buzzOnMotion && !silenced && alertMotion && armed) on = (now /
500) % 2 == 0;    // lent : intrusion (si activ  )
    buzzerWrite(on);
}

// =====
// D  marrage
// =====
void setup() {
    pinMode(PIN_BUZZER, OUTPUT);
    buzzerWrite(false);
    pinMode(PIN_PIR, INPUT);

    Serial.begin(115200);
    delay(50);
    Serial.printf("\n\nSentinel-X . %s . firmware %s\n", DEVICE_HOSTNAME,
FW_VERSION);

    snprintf(topicTelemetry, sizeof topicTelemetry, "%s/telemetry",
TOPIC_BASE);
    snprintf(topicAlert, sizeof topicAlert, "%s/alert", TOPIC_BASE);
    snprintf(topicStatus, sizeof topicStatus, "%s/status", TOPIC_BASE);
    snprintf(topicCmd, sizeof topicCmd, "%s/cmd", TOPIC_BASE);
    snprintf(topicConfig, sizeof topicConfig, "%s/config", TOPIC_BASE);

    //   cran : recherche de son adresse I2C (0x27 ou 0x3F le plus souvent)
    Wire.begin(PIN_SDA, PIN_SCL);
    uint8_t addr = LCD_ADDR;
    if (addr == 0) {
        const uint8_t candidates[] = {0x27, 0x3F, 0x20, 0x21, 0x22, 0x23, 0x24,
0x25, 0x26,
                                0x38, 0x39, 0x3A, 0x3B, 0x3C, 0x3D, 0x3E};
        for (uint8_t a : candidates) {
            if (i2cPresent(a)) { addr = a; break; }
        }
    }
}
```

```

if (addr) {
    lcd = new LiquidCrystal_I2C(addr, 16, 2);
    lcd->init();
    lcd->backlight();
    char l1[17];
    snprintf(l1, sizeof l1, "%s v%s", DEVICE_ID, FW_VERSION);
    lcdLine(0, "SENTINEL-X");
    lcdLine(1, l1);
    Serial.printf("[lcd] écran trouvé en 0x%02X\n", addr);
} else {
    Serial.println(F("[lcd] écran introuvable : vérifier SDA/SCL et le 5
V"));
}

dht.begin();

#if MQTT_USE_TLS
    // TLS : le boîtier ne fait confiance qu'à la CA du groupe
    caCert = new BearSSL::X509List(CA_CERT);
    net.setTrustAnchors(caCert);
#endif
    net.setTimeout(5000);
    // MQTT_HOST peut être un nom (sentinel.local) ou une adresse IP (ex. IP du
    PC sur le partage iPhone).
    // Avec une IP, le certificat reste vérifié par la CA du groupe, mais pas
    le nom inscrit dedans :
    // on peut changer de réseau sans régénérer les certificats.
    IPAddress brokerIp;
    if (brokerIp.fromString(MQTT_HOST)) mqtt.setServer(brokerIp, MQTT_PORT);
    else mqtt.setServer(MQTT_HOST, MQTT_PORT);
    mqtt.setCallback(onMqttMessage);
    mqtt.setBufferSize(768);
    mqtt.setKeepAlive(15);
    mqtt.setSocketTimeout(5);

    static WiFiEventHandler onWifiLost = WiFi.onStationModeDisconnected([]
(const WiFiEventStationModeDisconnected &e) {
        Serial.printf("[wifi] coupure, raison %d\n", e.reason);    // 2/15 = mot
de passe, 201 = réseau introuvable
    });
    (void)onWifiLost;
    WiFi.persistent(false);
    WiFi.mode(WIFI_STA);
    WiFi.hostname(DEVICE_HOSTNAME);
    WiFi.setSleepMode(WIFI_NONE_SLEEP);
    WiFi.setAutoReconnect(true);

```

```

WiFi.begin(WIFI_SSID, WIFI_PASS);
Serial.printf("[wifi] connexion à %s...\n", WIFI_SSID);

buzzerWrite(true);           // petit bip de démarrage
delay(120);
buzzerWrite(false);
}

// =====
//  Boucle principale : chaque tâche est rapide, rien ne bloque
//  =====
void loop() {
    uint32_t now = millis();

    netTask(now);
    mqtt.loop();
    pirTask(now);

    if (now - lastGasRead >= GAS_PERIOD_MS) {
        lastGasRead = now;
        readGas();
    }
    if (now - lastDhtRead >= DHT_PERIOD_MS) {
        lastDhtRead = now;
        readDht();
    }
    if (now - lastSensorRead >= TELEMETRY_PERIOD_MS) {
        lastSensorRead = now;
        publishTelemetry();
    }

    buzzerTask(now);
    lcdTask(now);
}

```